

POS-sible

Growth Intelligence Implementation Plan

A phased, additive roadmap to evolve a Bangladesh-first POS into a proactive business-growth platform—without destabilizing existing functionality.

Core principle

Protect the transactional core. Add intelligence around it.

Product vision

POS-sible should not only report what happened. It should help shop owners understand what is happening, decide what to do next, take action safely, and measure the result.

Audience: Bangladesh-first retailers and small businesses. Initial approach: transparent metrics and rules first; AI and advanced prediction later.

How to use this plan

Give coding agents one milestone at a time. Require an architecture audit before implementation, a narrow scope for each ticket, regression checks, and a clear report of files changed, migrations, tests, risks, and rollback steps.

1. Non-negotiable engineering principles

- **Preserve current behavior:** do not rewrite checkout, sales, inventory, payments, authentication, or existing reports as part of Growth work.
- **Prefer additive changes:** new modules, services, routes, tables, indexes, and UI pages. Use backward-compatible migrations; never delete or repurpose existing fields without an explicit migration plan.
- **Reuse the source of truth:** analytics must read existing transactional data. Do not create a second, independently updated copy of sales, stock, or customers.
- **Keep the transactional path fast:** expensive analytics, forecasts, and LLM calls must not run synchronously during checkout.
- **Make recommendations explainable:** show evidence, date range, confidence/limitations, and estimated impact. Do not present estimates as guaranteed revenue.
- **Human approval by default:** never automatically apply discounts, edit prices, contact customers, or change stock without an explicit user action.
- **Tenant/store isolation:** every query, cache key, job, and AI tool must enforce the current business/store scope.
- **Feature flags and rollback:** Growth modules should be switchable off without disabling core POS operations.

Required workflow for every agent

- Inspect the existing repository and summarize the current architecture before changing code.
- Identify exact routes, models, services, and APIs to reuse; do not assume a stack or invent endpoints.
- Write a small implementation plan and list likely regressions before editing.
- Implement only the requested milestone; avoid unrelated refactors and broad formatting changes.
- Add or update tests, run existing checks, and report failures honestly.
- Provide migration/rollback instructions and manual QA steps for the affected flows.

2. Target architecture

Layer	Responsibility	Boundary
Existing POS core	Sales, checkout, payments, inventory, customers, authentication	Remain source of truth; minimize changes
Analytics/read layer	Aggregated metrics, trend windows, product/customer summaries	Read-only at first; cache/precompute if needed
Rules & detection	Low stock, dead stock, target risk, customer risk, sales changes	Deterministic rules with documented thresholds
Growth domain	Opportunities, recommendations, accepted actions, experiments	Own only Growth-specific records
AI interface (later)	Natural-language questions and summaries using approved tools	Never query arbitrary DB or invent business facts

Data flow

Existing transactions → analytics/read models → deterministic detection → opportunity/recommendation → user-approved action → outcome measurement → AI interface.

Do not introduce event sourcing just for this roadmap. If an event/audit mechanism is useful, add lightweight events or audit records around existing flows without changing their semantics.

3. Phased implementation roadmap

Build in dependency order. A phase is complete only when its acceptance criteria pass—not merely when the UI looks finished.

Phase 0 — Audit, baseline, and safety

Goal: Understand the existing app and establish a regression baseline.

- Map frontend routes/components, backend services/APIs, database schema, authentication/authorization, and current Growth implementation.
- Document critical flows: sign in, checkout/sale completion, cancellation/refund if supported, stock updates, customer operations, payment recording, reports.
- Run existing build/tests and record baseline results; add targeted regression tests where coverage is missing.
- Identify tenant/store ownership rules, timezone/currency handling, and existing definitions of revenue, cost, margin, refunds, and cancelled sales.
- Create a feature flag or safe route-level boundary for new Growth functionality.

Deliverable: Architecture note, data dictionary, regression checklist, baseline test report, and a list of safe extension points.

Trade-off: Do not perform a broad rewrite or refactor during the audit.

Phase 1 — Analytics foundation

Goal: Create reliable metrics from existing transactional records.

- Define canonical calculations for gross/net sales, order count, average order value (AOV), discounts, returns/refunds, cost, and gross profit where source data supports them.
- Add read-only query services and, only where needed, aggregate views/tables for daily sales, product sales, customer purchase history, stock velocity, margin, and payment methods.
- Ensure all metrics respect store/business scope, date range, timezone, and existing business rules.
- Add indexes or caching only after identifying query bottlenecks; never slow down checkout to calculate Growth metrics.

Deliverable: Documented metric definitions, tested read APIs/services, and a comparison against existing reports for the same date range.

Trade-off: Aggregates improve speed but can become stale; begin with direct queries or controlled refreshes and expose freshness where relevant.

Phase 2 — Growth targets and scenarios

Goal: Make the existing target feature useful without changing how sales are recorded.

- Show target, actual sales, elapsed/remaining days, required daily sales, current pace, projected month-end result, and projected gap.
- Classify pace as ahead, on track, at risk, or behind using explicit thresholds.
- Add a scenario simulator for changes to customer count, order count, AOV, and discount level.
- Clearly label projections as estimates; handle new stores, incomplete history, closed days, and partial months.

Deliverable: Growth overview and target calculations with unit tests for edge cases.

Trade-off: Use simple transparent formulas before statistical models. Avoid false precision and never guarantee outcomes.

Phase 3 — Opportunity model and rules engine

Goal: Create a reusable framework for business insights.

- Add a Growth-owned Opportunity model with type, title, description, severity, evidence, estimated impact, confidence, related entity, status, created/expiry timestamps, and deduplication key.
- Implement deterministic rules for target risk, sales spikes/drops, low stock, dead stock, margin decline, and high discount usage.
- Make rules configurable where thresholds vary by business type; document default thresholds.
- Deduplicate repeated findings, expire stale opportunities, and allow users to dismiss or mark them resolved.

Deliverable: Opportunity API/service, rules registry, and an Opportunities UI with evidence and status.

Trade-off: Rules are easier to debug than an LLM but can generate noise; tune thresholds and track dismissals.

Phase 4 — Inventory health and stockout estimates

Goal: Help owners decide what to restock or review.

- Calculate units sold over 7/30/90 days, last-sale date, sales velocity, current stock, and estimated days of inventory where data permits.
- Classify items as fast-moving, healthy, slow-moving, or possibly dead stock with configurable thresholds.
- Estimate stockout dates using a simple baseline such as a weighted moving average; later add day-of-week or seasonal adjustments.
- Show assumptions and confidence limitations; account for missing sales history, supplier lead time if available, and products with irregular demand.

Deliverable: Inventory health view, dead-stock opportunities, and stockout-risk warnings.

Trade-off: Start with explainable heuristics, not ML. Avoid recommending clearance purely from slow sales when items may be seasonal or intentionally stocked.

Phase 5 — Customer intelligence

Goal: Identify customer segments and potential retention opportunities.

- Calculate last purchase, purchase frequency/interval, order count, average order value, and total spend from existing customer-linked sales.
- Build segments such as new, active, loyal, at-risk, lost, and high-value with transparent criteria.
- Detect customers whose time since last purchase exceeds their historical interval, where enough history exists.
- Handle anonymous/walk-in sales and sparse history without inventing customer identities or lifetime value.

Deliverable: Customer segments and at-risk customer lists with explainable criteria.

Trade-off: Avoid treating every inactive customer as lost. Customer-level analysis only works when transactions are reliably linked to a customer.

Phase 6 — Product relationships and business health

Goal: Surface cross-sell ideas and a transparent summary of business condition.

- Use order-level co-occurrence to identify products frequently bought together; start with minimum support and confidence thresholds.
- Create cross-sell/bundle opportunities only when data volume is sufficient and stock/margin constraints are considered.
- Build a Business Health Score from documented component scores (sales, inventory, customers, profitability, target progress).
- Show each component's reason and the data period used; mark unavailable dimensions as unavailable rather than silently assigning a misleading score.

Deliverable: Product relationship insights and an explainable Business Health Score.

Trade-off: Basket analysis can mislead with low volume or popular-item bias. The score is a diagnostic aid, not an objective measure of business quality.

Phase 7 — Daily briefing and recommendations

Goal: Turn raw metrics and opportunities into prioritized next steps.

- Build a daily briefing endpoint/service with yesterday's sales, same-weekday comparison where possible, target pace, top/weak products, stock risks, and customer opportunities.
- Create a Recommendation model that references evidence/opportunities, estimated impact, confidence, priority, action type, and status.
- Rank recommendations by impact, urgency, confidence, and actionability; avoid duplicate or contradictory suggestions.
- Make every recommendation link to the relevant product/customer/target page and explain why it appeared.

Deliverable: Growth landing page: today's briefing, top priorities, and an evidence-backed opportunity list.

Trade-off: Keep prioritization deterministic at first. LLM-generated prose may summarize verified data but must not invent numbers.

Phase 8 — Safe action framework

Goal: Let users act on recommendations without bypassing existing business rules.

- Define action types and route each action through existing validated services/APIs.
- Begin with navigation and draft actions: view products, open customer segment, create a draft promotion, or adjust a target.
- Require confirmation for consequential actions such as price/discount changes, stock changes, and customer messaging.
- Record who initiated an action, when, what changed, and whether it succeeded; make actions idempotent where retries are possible.

Deliverable: Recommendation-to-action flow with confirmation, audit trail, and clear failure states.

Trade-off: Never let the Growth module write directly to transactional tables or bypass current validation and permissions.

Phase 9 — Measure outcomes and experiments

Goal: Learn whether recommendations and campaigns produce useful results.

- Record recommendation viewed, accepted, dismissed, action started, and action completed.
- Compare before/during/after metrics over explicit windows and account for seasonality, refunds, and margin—not revenue alone.
- Add an experiment model for a hypothesis, baseline, intervention, time window, metric, and result.
- Describe results as observed changes, not proven causation, unless a suitable control/experiment design supports causal claims.

Deliverable: Recommendation outcome tracking and a simple experiment dashboard.

Trade-off: A before/after increase does not prove the recommendation caused it. Use careful wording and control groups only when practical.

Phase 10 — AI Business Assistant

Goal: Enable natural-language questions grounded in verified business data.

- Expose a small, permission-checked set of structured tools: sales summary, product performance, inventory health, customer segments, target progress, opportunities, and profit summary.
- Let the model choose among approved tools, then explain results using returned data and clearly state gaps/assumptions.
- Enforce store scope server-side; never trust a store ID or authorization claim supplied by the model.
- Add timeouts, rate limits, token/cost controls, safe error handling, and a no-AI fallback to the regular dashboard.
- Prevent the model from executing destructive actions or arbitrary SQL. Keep action execution in validated backend services with user confirmation.

Deliverable: AI assistant for questions such as “Why did sales drop?” and “What should I restock?” with citations/evidence to app data.

Trade-off: An LLM is a reasoning/interface layer, not the source of truth. Ship only after metric definitions and permissions are reliable.

Phase 11 — Bangladesh-specific enhancements

Goal: Make the intelligence relevant to local retail patterns.

- Add optional seasonal analysis for Ramadan, Eid, local holidays, and weekday patterns when enough historical data exists.
- Analyze payment-method mix (cash, bKash, Nagad, cards, other) only where the app records it reliably.
- Consider Bangla-friendly summaries, BDT-aware formatting, and low-bandwidth/mobile usability.
- Treat WhatsApp/social messaging as a separate integration and campaign module; support draft, preview, consent/opt-out rules, and explicit send confirmation.

Deliverable: Locally relevant forecasts and campaign drafts, gated by data quality and integration readiness.

Trade-off: Seasonal patterns vary by store and category; do not hardcode one retailer's behavior as universal.

4. Suggested delivery tickets

Priority	Ticket	Acceptance criteria
P0	Architecture audit + data dictionary	No production behavior changes; baseline build/tests documented.
P0	Regression tests for critical POS flows	Checkout/sales/inventory/customer/auth/report behavior protected.
P1	Canonical analytics metrics	Metric definitions documented and validated against existing reports.
P1	Growth target calculations	Correct edge cases; no writes to sales or inventory.
P1	Opportunity model + rules registry	Deduplication, expiry, status, tenant isolation, unit tests.
P1	Inventory health + dead-stock rules	Evidence, thresholds, sparse-history handling.
P2	Stockout estimates	Assumptions visible; forecast isolated behind service interface.
P2	Customer segments + at-risk detection	Only customer-linked data; no false precision for sparse history.
P2	Product co-purchase analysis	Minimum data thresholds and margin/stock guardrails.
P2	Business Health Score	Transparent components and unavailable-data handling.
P2	Daily briefing + recommendation ranking	Deterministic ranking; actionable deep links.
P3	Action framework + audit log	Validated existing services, confirmations, idempotency.
P3	Outcome tracking + experiments	Clear windows; no unsupported causal claims.
P4	Structured AI tools + assistant	Permission-checked tools; no arbitrary SQL or direct writes.
P5	Local seasonality + messaging integrations	Only when data quality, consent, and provider requirements are met.

5. Definition of done for every ticket

- Existing core flows still work: sign-in, checkout, sale completion, stock updates, customer flows, payments, and current reports.
- No unexplained breaking API/schema changes; migrations are backward-compatible and include rollback notes where feasible.
- Authorization and store/business isolation are tested, including attempts to access another store's data.
- Tests cover normal paths, empty data, missing history, date boundaries, refunds/cancellations where supported, and permission failures.
- Build/lint/type checks and relevant tests are run; results and any pre-existing failures are reported accurately.
- UI includes loading, empty, error, stale-data, and permission states; accessible labels and responsive behavior are preserved.
- Agent reports: summary, files changed, schema changes, API changes, tests run/results, manual QA, risks, and rollback.

6. Risks and decisions to keep explicit

Risk	Mitigation
Metric mismatch	Document canonical definitions; compare with current reports; include refunds/discounts consistently.
Slow dashboards	Profile queries, index appropriately, cache/aggregate only when justified; keep analytics off checkout.
Noisy recommendations	Thresholds, deduplication, expiry, dismiss controls, and outcome tracking.

Risk	Mitigation
False predictions	Baseline heuristics, visible assumptions, confidence limits, no guarantees.
AI hallucination/data leakage	Structured tools, server-side authorization, minimal context, no arbitrary SQL, audit and rate limits.
Breaking existing behavior	Small PRs, regression tests, feature flags, backward-compatible migrations, rollback plan.
Overbuilding	Ship Phase 0–3 first; validate with real shop owners before committing to advanced ML or messaging.

7. Copy-paste master instruction for coding agents

Use the following as the shared preamble for every implementation ticket:

You are working on POS-sible, an existing POS application. Implement only the specific ticket provided after this instruction.

SAFETY AND SCOPE

- First inspect the current codebase, architecture, relevant routes/services/models, and existing tests.
- Do not assume the stack, schema, API names, or business rules; verify them in the repository.
- Do not rewrite or alter existing checkout, sales, inventory, customer, authentication, payment, or reporting behavior unless the ticket explicitly requires it.
- Prefer additive, isolated changes. Reuse existing source-of-truth data and validated services. Do not duplicate transactional data.
- Keep analytics and AI work out of synchronous checkout paths.
- Enforce authorization and business/store/tenant isolation on the server for every query and action.
- Do not allow an LLM to execute arbitrary SQL, bypass validation, or perform destructive actions. Require user confirmation for consequential actions.
- Add feature flags where appropriate. Use backward-compatible database migrations; do not delete or repurpose fields casually.

WORKFLOW

1. Summarize the relevant architecture and propose the smallest safe implementation.
2. List risks and regression cases before coding.
3. Implement only this ticket; avoid unrelated refactors.
4. Add/update tests for normal, empty, edge, date-boundary, permission, and failure cases relevant to the change.
5. Run available tests/build/lint/type checks. Never claim checks passed unless they ran.
6. Provide migration and rollback notes, manual QA steps, and any known limitations.

DEFINITION OF DONE

- Existing core flows remain functional.
- New behavior is scoped, explainable, and handles loading/empty/error states.
- Tests and build results are reported honestly.
- Final response lists changed files, schema/API changes, checks run and results, risks, and rollback steps.

TICKET:

[Paste exactly one implementation ticket here.]

Recommended first move

Give an agent only Phase 0. Review its architecture findings and regression baseline before authorizing Phase 1. This is the safest way to make the product more ambitious without risking the POS that already works.

Planning note: this roadmap is intentionally technology-agnostic. Final implementation details should be adapted to the repository's actual architecture and current data model.